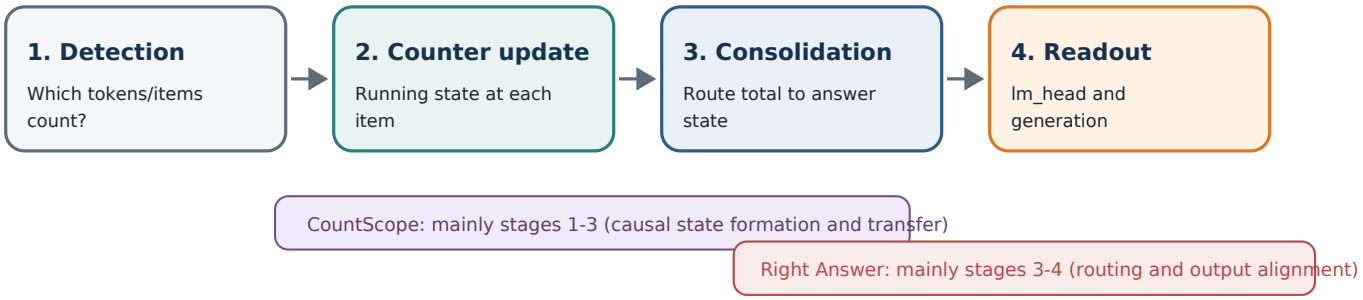
Latent Counters, Routing, and Readout

How two recent counting papers complement each other - and what their combined account predicts for long-context NIAH experiments

Executive thesis

The papers are largely complementary. Understanding Counting Mechanisms in Large Language and Vision-Language Models explains how a latent count is formed, updated, localized, and transferred inside the network. The Right Answer, the Wrong Direction explains why an already-decodable count can still fail to become the correct digit or generated answer. Together they imply that counting can fail at two separable bottlenecks: counter formation/consolidation and representation-to-output readout.

A unified four-stage account



The distinction is operational, not merely verbal. A model can encode occurrence index at item tokens, transfer that latent state under activation patching, and still assign low probability to the correct digit because the final answer state and the output rows are poorly aligned.

The two papers at a glance

Paper	Primary question	Primary evidence	Main bottleneck
CountScope / Counting Mechanisms [1]	How is a count built and stored across tokens and layers?	PCA/cosine analyses, causal mediation, zero/mean/interchange patching, and CountScope transfer into a controlled target context.	Limited or shortcut-prone counter formation; depth, item type, gaps, separators, and layout affect the latent state.
Right Answer, Wrong Direction [2]	Why can a decodable count still yield the wrong output?	Ridge probes, logit lens, probe/output-row cosine geometry, digit-row repair, DPS, and LoRA Q/V.	Geometric and routing mismatch between count features, final answer states, and digit-token output rows.

Bottom line for NIAH: before concluding that the model lacks a count feature, separately test (a) whether marker/item states carry a running count, (b) whether the last marker or final prompt token consolidates the total, and (c) whether the model's own output head reads that state into the correct count token.

1. Paper A: the internal-counter account

Understanding Counting Mechanisms in Large Language and Vision-Language Models (Hasani et al., CVPR 2026) studies repeated textual items and visual objects, using Qwen2.5/Llama-family LLMs and Qwen2.5-VL/InternVL-family LVLMs. The main textual tasks are controlled lists of 1-9 items, in monotypic and polytypic conditions, with list-first and question-first prompt orders. [1]

What CountScope adds

Standard Logit Lens and Tuned Lens were reported as too noisy for numerical concepts in this setup. CountScope instead patches a source activation into a placeholder in a minimal target counting context, then uses the target model's numerical output distribution as a causal decoder of the source activation. [1, Sec. 4.1]

Key mechanistic findings

- Per-item latent state: the hidden state at the j-th item can decode as count j, producing an item-by-item trajectory rather than a count signal that appears only at the final answer token.
- Continued counting: patching final source items into the beginning of a target list can make the target continue from the source latent count. The authors characterize this as a memoryless or Markov-like update based on the most recent latent state.
- Final-item concentration: the final contextual item/token often has the strongest causal influence on the reported total, although the paper also reports a context-level maximum-latent-count effect rather than a purely local final-token rule.
- Layerwise emergence: smaller count values become available in lower layers, while larger values require deeper layers. The paper interprets this as a depth-linked capacity limit.
- Type specificity and reset: in polytypic lists, separate counters can form for different item types and may reset after sufficiently long interruptions.
- Separator shortcuts: commas and other separators can carry stronger positional-count information than the item tokens themselves; disturbing separator structure damages counting.
- Approximate linear additivity: position-difference vectors can shift the decoded count of an item state toward another position, suggesting a partly disentangled numerical direction.

What the paper does not establish

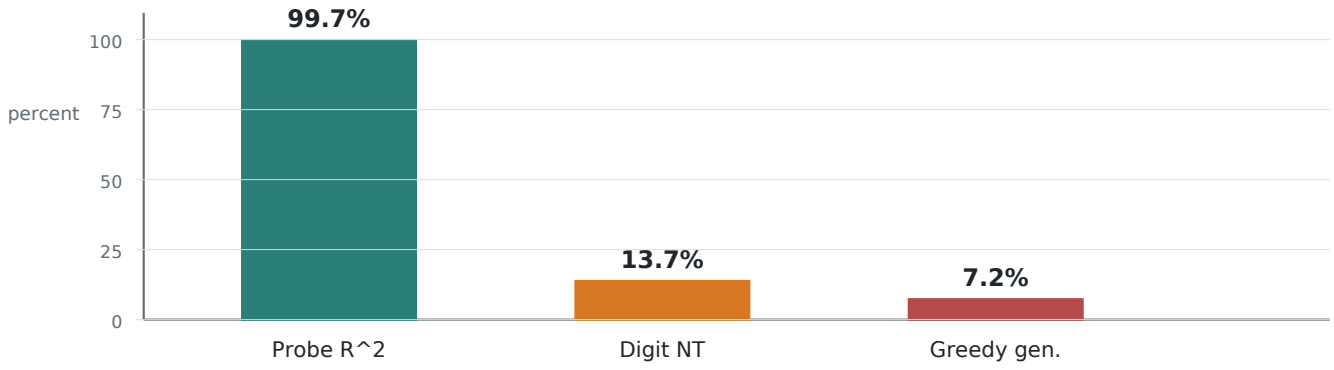
The evidence supports a transferable latent state that behaves like a counter, but it is not yet a complete circuit-level description. "Counter" should not be read as proof of one scalar neuron, one universal direction, or a task-independent exact algorithm. The authors explicitly leave full circuit discovery and chain-of-thought counting open. [1, Sec. 6]

Important scope difference for your work

The paper's lists are short and structurally regular. Paul Graham haystacks with sparse markers add long-range retrieval, heterogeneous local contexts, and irregular spacing. Therefore, failure to reproduce a clean latent trajectory in NIAH would not directly refute CountScope; it may show that detection and counter updates become unreliable before readout.

2. Paper B: the geometric-readout account

The Right Answer, the Wrong Direction (Garcia, arXiv v2, May 2026) studies controlled entity counting and related low-vocabulary aggregation tasks across Qwen3, Mistral, and Pythia models. Its strongest Qwen3-8B result is a large gap between what an external probe can decode and what the model emits. [2]



Illustrative Qwen3-8B entity-counting results from the harmonized evaluation: entity-mean ridge probe R^2 reaches 0.997, while the baseline is 13.7% for digit-restricted next-token prediction and 7.2% for greedy generation. These quantities use different metrics, so the chart visualizes the representation-output gap rather than a like-for-like accuracy comparison. [2, Tables 1-2; Fig. 2]

Core diagnosis

- Count is linearly decodable: ridge probes trained on entity-mean residual activations achieve R^2 above 0.99 across layers in the controlled task.
- The model's own decoder is misaligned: probe directions have near-random cosine alignment with digit-token rows of the lm_head (reported |cos| <= 0.032).
- Training-distribution explanation: a digit row is trained across dates, identifiers, list labels, arithmetic, and many other contexts; counting contexts need not dominate the conditional hidden-state average that shapes that row.
- Vocabulary competition matters: digit rows have relatively low output-row norms, so punctuation, whitespace, and other tokens can beat the correct digit in full-vocabulary generation.
- Routing is part of the deployment problem: repairing only nine digit rows substantially improves constrained next-token decoding but achieves 0% in the paper's unconstrained greedy-generation setting.

Interventions and what they imply

Method	Target	Result / interpretation
Probe-round	External linear readout	Near-ceiling diagnostic upper bound; shows information is recoverable, not that the model naturally uses it.
9-row lm_head repair	Digit output rows	60.7% digit-restricted and 60.3% full-vocab next-token on Qwen3-8B entity counting, but 0% greedy generation in the reported setting.
Hard / soft DPS	Decode-time logits	Hard bypass is strong diagnostically; soft steering remains near baseline, showing that small logit nudges do not solve routing and vocabulary competition.
LoRA Q/V rank-16	Upstream attention routing	83.1% +/- 7.2% greedy generation in the multi-task setting; improves final-layer readability without rewriting the early count direction.

3. How the papers complement each other

Combined mechanistic claim

CountScope: a transformer can build an item-indexed latent state and transfer it causally. Right Answer: even a high-fidelity latent count can remain behaviorally inert when it is not routed to an answer state or aligned with the vocabulary rows needed to express it.

Question	CountScope answer	Right Answer answer	Joint interpretation
Where does count live?	At successive item/separator states and especially late contextual positions.	Linearly recoverable from entity-mean states; partly available at the final prompt position.	Count can be distributed locally, then consolidated into a readout state.
How does it evolve?	Incremental, position-dependent, layerwise, transferable, and approximately additive.	Stable linear direction across depth; later layers attempt to make it output-compatible.	Formation and vocabulary alignment are separate transformations.
Why can behavior fail?	Counter resets, depth limits, type interference, and reliance on separators/structure.	Probe direction is orthogonal to digit rows; output norms and routing cause competition.	Errors may arise before or after a correct latent count exists.
What intervention helps?	Activation transfer and position-difference additions change decoded counts.	Readout repair helps constrained decoding; Q/V LoRA helps generation.	Intervene at the stage matching the diagnosed bottleneck.

Apparent contradictions that are not contradictions

1

"The final token stores the count" vs. "the model predicts the wrong digit"

A hidden state may contain count information that an external patching decoder or probe can recover while the model's own lm_head remains poorly aligned. Storage is not equivalent to native readout.

2

"Logit Lens is too noisy" vs. extensive logit-lens analysis

CountScope found ordinary lenses unreliable for decoding intermediate item-level numerical concepts and therefore used a controlled target context. Right Answer uses the lens as a test of the model's own output alignment. A noisy lens is precisely what geometric misalignment predicts.

3

"Larger counts require deeper layers" vs. high ridge R^2 at nearly every layer

The measurements differ. CountScope asks when a specific count is causally decodable from an actual item state. Right Answer pools entity positions into a synthetic entity-mean representation and asks whether a supervised linear map can recover the example-level count. Pooling and supervision can expose information before the model has consolidated it at a real generation position.

4

"Counter state is additive" vs. "count direction is orthogonal to digits"

Additivity describes geometry within the residual stream. Orthogonality describes the relationship between that geometry and the output matrix. Both can hold simultaneously.

4. Important differences in experimental meaning

Dimension	CountScope [1]	Right Answer [2]	Why this matters
Model family	Main text emphasizes Qwen2.5-7B and Qwen2.5-VL-7B; additional Llama/InternVL models.	Qwen3-8B, Mistral-7B, Pythia-410M, plus scale extensions.	A direct Qwen3 replication of CountScope is still needed for your setup.
Input regime	Regular lists of 1-9 items and controlled visual objects.	Short synthetic paragraphs with counts, distractors, lengths, and spacing factors; low-vocabulary answers.	Neither paper directly establishes the same mechanism in sparse 1K-32K NIAH contexts.
Readout object	Actual item/separator states decoded by transfer into a minimal target context.	Entity-mean synthetic aggregate and actual final prompt state decoded by probes/lm_head.	Entity-mean R^2 is not proof that any one forward-pass token carries the full count.
Causal claim	Patching changes latent decoded count and output probabilities.	Row repair and Q/V LoRA localize output/routing bottlenecks.	One paper identifies the state update; the other identifies the expression failure.
Failure mode	Reset, maximum-latent-count behavior, separator shortcuts, depth limits.	Output-row orthogonality, low digit-row norms, full-vocab competition, incomplete routing.	A single wrong answer can have multiple mechanistically distinct causes.

Strongest synthesis

A robust counting system needs both a state machine and an interface. The state machine must detect each countable event, update a type-appropriate latent count, avoid resets, and preserve the total. The interface must route that total to the answer position and align it with the correct output token under full-vocabulary competition. CountScope primarily reveals the state machine; Right Answer primarily diagnoses the interface.

A caution about the word "knows"

High probe performance establishes decodability under a particular representation and probe class. It does not by itself establish that the model uses the same variable in its native computation. CountScope strengthens the causal case by transferring activations and measuring downstream changes; Right Answer strengthens it by repairing the output rows and routing weights. Even together, they do not yet identify a complete, unique counting circuit.

A caution about noise

The papers use "noise" in different senses. CountScope says ordinary lens decoding of intermediate numerical concepts can be noisy. Right Answer shows that supervised probes can nevertheless achieve very high R^2 in its controlled task, while same-count hidden states still exhibit nuisance variation that makes full-vocabulary repair harder. Thus, decodability, native readability, and within-class compactness should be measured separately.

5. Predictions for the [dolphin] NIAH task

Your marker-counting setup adds a long natural-text haystack and sparse repeated markers. The combined papers suggest four diagnostic questions, ordered by causal stage.

1

Occurrence detection

Does each [dolphin] span produce a reliable marker-specific state, independent of local Paul Graham text? Compare marker tokens with matched non-marker controls and inspect false-negative examples.

2

Running counter

At the j-th occurrence, is j decodable? Fit occurrence-index probes or use clean-to-corrupt patching between occurrence states. Check marker token, closing bracket, newline, punctuation, and post-marker positions.

3

Total consolidation

Is the gold count decodable from the last marker, its following separator, the final context token, the query token, and the final prompt token? This distinguishes local counting from answer-time routing.

4

Native readout

At positions with high probe accuracy, apply final RMSNorm plus lm_head and score count candidates. Measure correct-count rank, margin, and alignment between probe directions and digit/JSON-token rows.

Interpretation matrix

Observed pattern	Most likely bottleneck	Paper connection
Occurrence-index probe is weak	Marker detection or local update is unreliable; long-context/local-context variation dominates.	Earlier than the clean latent-counter regime of CountScope.
j is decodable at markers, but total is weak at final token	Counter exists locally but is not consolidated or routed.	CountScope-like counter; pre-readout routing failure.
Total probe is strong at final token, but candidate count rank is poor	Representation-output geometry is misaligned.	Closest to Right Answer's readout bottleneck.
Correct count ranks high, but greedy JSON is wrong	Autoregressive formatting or token-competition failure after initial readout.	Explains why row repair may fail in generation.
Separator states outperform marker states	The model is exploiting structural rhythm rather than robust object enumeration.	CountScope separator-shortcut result.

6. A high-value experimental sequence

The following sequence minimizes interpretive ambiguity and reuses the same examples, hidden states, and token-span metadata.

Step	Analysis	Primary metric	Decision enabled
A	Behavioral count ladder over counts 0-10, multiple haystack lengths, and matched prompt formats.	Accuracy, MAE, undercount/overcount, candidate-count logprob.	Separates count magnitude, length, and output-format effects.
B	Occurrence-index decoding h_(i,j) -> j at marker, bracket, separator, and post-marker positions.	Held-out R^2, rounded accuracy, monotonicity.	Tests whether a CountScope-like running counter exists.
C	Per-example total-count probes at marker mean, last marker, post-marker, final context, query, and final prompt.	Held-out R^2 and count-class accuracy by layer.	Locates consolidation and routing.
D	Clean/corrupt activation patching: change one marker occurrence and restore selected states.	Recovery of gold-count margin and count probability.	Tests causal necessity/sufficiency of local and final states.
E	Native readout geometry at high-decoding positions.	Correct token rank, logit margin, cosine to digit/JSON rows.	Tests Right Answer-style output misalignment.
F	Intervention matched to diagnosis: occurrence/route patch, Q/V LoRA, or constrained output-row repair.	Held-out greedy accuracy and collateral damage.	Distinguishes diagnostic rescue from deployable fix.

Three controls that are especially important

- Separator control: keep marker count fixed while changing newline/comma/sentence-boundary structure; separately patch marker and separator states.
- Scope control: prevent instruction/query copies of [dolphin] from being mistaken for countable context occurrences; report prompt-wide and context-only marker counts.
- Generalization control: train probes/interventions on one set of haystacks, marker positions, and count values, then test on held-out haystacks, positions, lengths, and ideally held-out counts.

Most informative possible result

If occurrence index is causally decodable at marker or separator states, the total is strongly decodable at the last marker/final prompt state, but the model's candidate-count distribution still favors the wrong digit, your NIAH task would reproduce both papers in one pipeline: a CountScope-like latent counter plus a Right Answer-style output bottleneck.

7. Conclusions and research cautions

Complement, not contradiction. CountScope provides causal evidence that ordered item states can carry a transferable, incrementally updated latent count. Right Answer shows that a linearly decodable count need not be aligned with the model's output rows or generation pathway. The two claims occupy adjacent stages of the same computation.

The main unresolved bridge is consolidation. CountScope focuses on item-level and final-context states; Right Answer focuses on entity-mean and final-prompt readout. A decisive synthesis experiment should trace the same count information from the j-th item, to the last item/separator, to the query/final prompt token, and finally through lm_head.

Do not infer readout failure from behavior alone. A Right Answer-style claim requires high held-out internal decodability at an actual answer-driving position or a carefully justified aggregate, plus evidence that native output logits remain misaligned. Low probe performance points instead to detection, state-update, or consolidation failure.

Do not infer a universal counter from one regular list format. Separator shortcuts, type-specific resets, depth dependence, prompt order, and local context can all alter the mechanism. Sparse long-context NIAH is a valuable stress test precisely because it removes the regular rhythm that may make short-list counting easy.

References

[1] H. Hasani, A. Izadi, F. Askari, M. Bagherian, S. Mohammadian, M. Izadi, and M. Soleymani Baghshah. Understanding Counting Mechanisms in Large Language and Vision-Language Models. CVPR 2026; arXiv:2511.17699v2. Official open-access paper: CVF PDF.

[2] G. Garcia. The Right Answer, the Wrong Direction: Why Transformers Fail at Counting and How to Fix It. arXiv:2605.03258v2, 15 May 2026. arXiv record.

Source note

This memo synthesizes the papers' stated experiments and adds an explicit four-stage framework for comparison. Claims about how the papers fit together are interpretive inferences, not claims made verbatim by either paper. The comparison uses the CVPR 2026 open-access version of [1] and arXiv v2 of [2].

One-sentence synthesis

Transformers can maintain a latent counter, yet still fail to count behaviorally because the counter may be incomplete, reset, poorly consolidated, or geometrically unreadable by the output pathway.